Computer Organization & Architecture

Unit- 6
Central Processing Unit
(Part - 1)

Prepared By: Sweetu D. Sureja
Information Technology Department
V.V.P. Engineering College, Rajkot.

Outline….

- Introduction
- General Register Organization
- Stack Organization
- Instruction format
- Addressing Modes
- Data transfer and manipulation
- Program control
- Reduced Instruction Set Computer (RISC) & Complex Instruction Set Computer (CISC)

Introduction

- **The part of the computer that performs the bulk of data-processing operations** is called the central processing unit.
- The CPU is made up of **three major parts**,
- The **register set stores intermediate data used during the execution of the instructions.**
- The arithmetic logic unit **(ALU) performs the required micro operations for executing the instructions.**
- The control unit **supervises the transfer of information** among the registers and instructs the ALU as to which operation to perform.

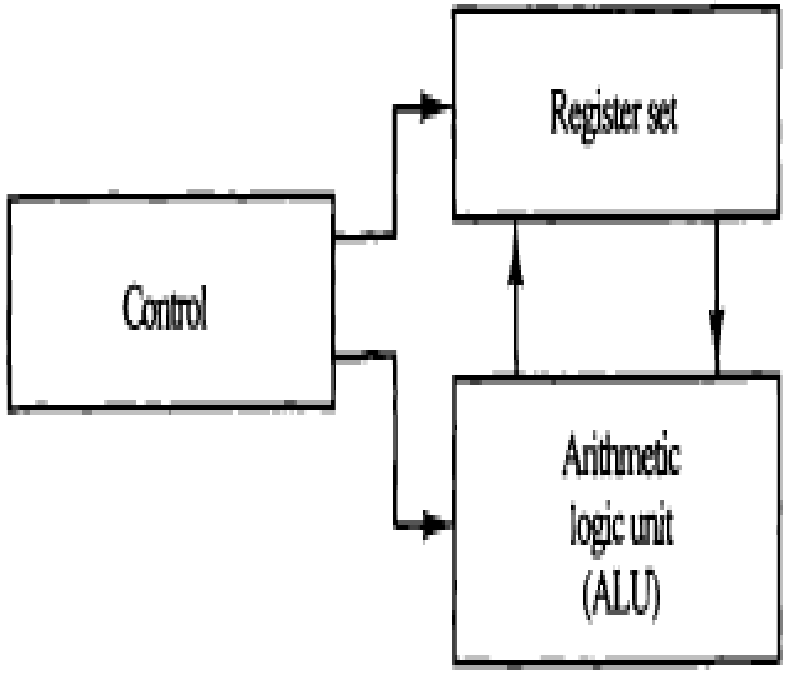


Figure 8-1 Major components of CPU.

- Processor organization also divided into 3 types:-

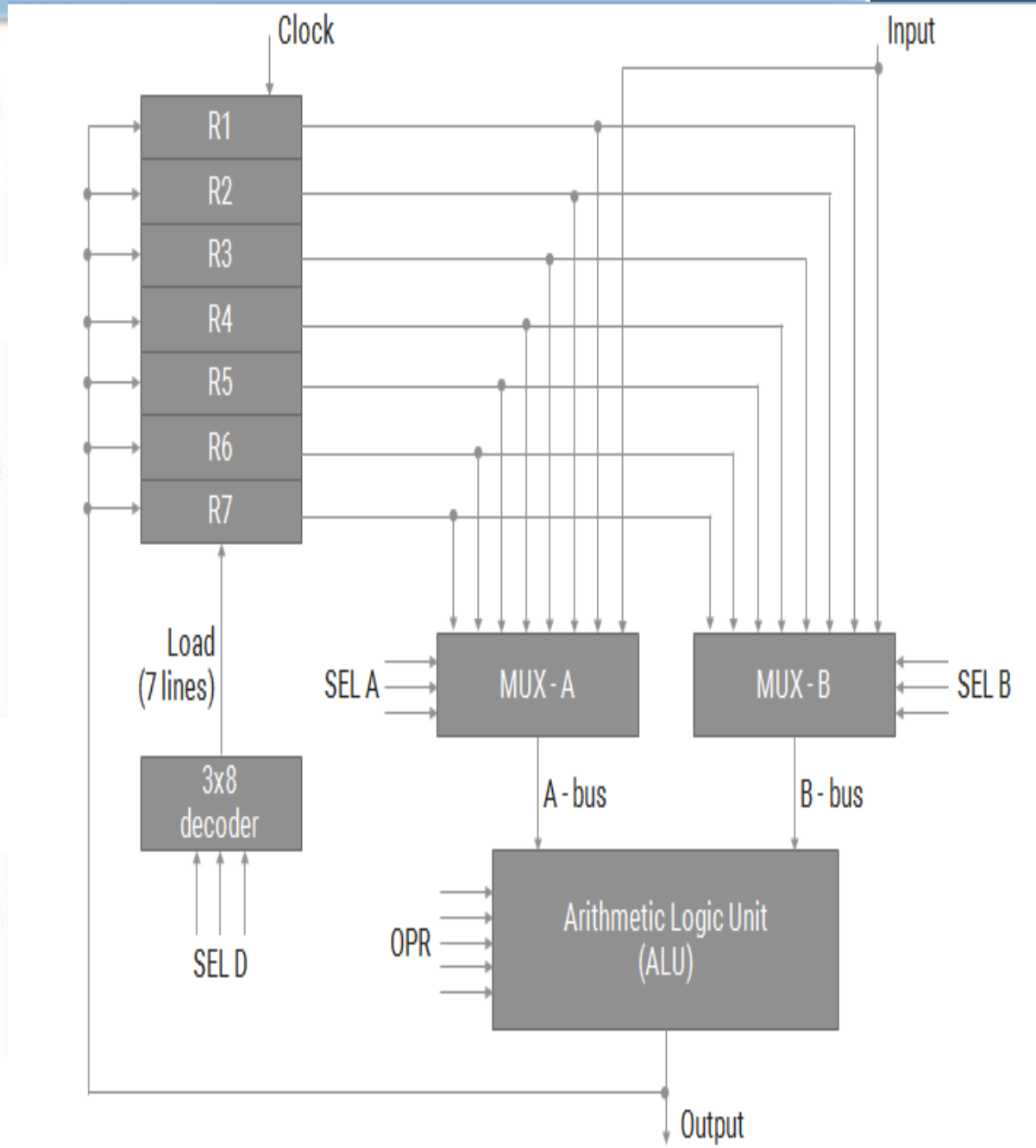
1) Single Accumulator Organization – Slow speed to execute Program and also line of code is more….

2) General Register Organization -

3) Stack Organization

General Register Organization

- The output of each register is connected to two multiplexers (MUX) to form the two buses A and B.
- The selection lines in each multiplexer-select one register
- The A and B buses form the inputs to a common arithmetic logic unit (ALU).
- The ALU determines the arithmetic or logic microoperation.
- The register that receives the information from the output bus is selected by a decoder.
- The decoder activates one of the register load inputs



General Register Organization

- Example: $R1 \leftarrow R2 + R3$
- To perform the above operation, the control must provide binary selection variables to the following selector inputs:

1. MUX A selector (SELA): to place the content of R2 into bus A.
2. MUX B selector (SELB): to place the content of R3 into bus B.
3. ALU operation selector (OPR): to provide the arithmetic addition A + B.
4. Decoder destination selector (SELD): to transfer the content of the output bus into R1.

- *Control Word:*

SELA	SELB	SELD	OPR

THREE ADDRESS FORMATE MICRO INSTRUCTION

E = (A+B)*(C+D)

ADD R1,A,B

ADD R2,C,D

MUL X,R1,R2

Control Word

Binary Code	SELA	SELB	SELD
000	Input	Input	None
001	R1	R1	R1
010	R2	R2	R2
011	R3	R3	R3
100	R4	R4	R4
101	R5	R5	R5
110	R6	R6	R6
111	R7	R7	R7

- There are 14 binary selection inputs in the unit, and their combined value specifies a control word.
- It consists of four fields. Three fields contain three bits each, and one field has five bits.
- The three bits of SELA select a source register for the A input of the ALU. The three bits of SELB select a register for the B input of the ALU.
- The three bits of SELD select a destination register using the decoder and its seven load outputs. The five bits of OPR select one of the operations in the ALU.

Cont...

Control Word

- The OPR field has five bits and each operation is designated with a symbolic name.

OPR Select	Operation	Symbol
00000	Transfer A	TSFA
00001	Increment A	INCA
00010	A + B	ADD
00101	A – B	SUB
00110	Decrement A	DECA
01000	A and B	AND
01010	A or B	OR
01100	A xor B	XOR
01110	Complement A	COMA
10000	Shift right A	SHRA
11000	Shift left A	SHLA

Encoding of ALU Operations

Examples of Micro operations

$R1 \leftarrow R2 - R3$

Field:	SELA	SELB	SELD	OPR
Symbol:	R2	R3	R1	SUB
Control word:	010	011	001	00101

TABLE 8-3 Examples of Microoperations for the CPU

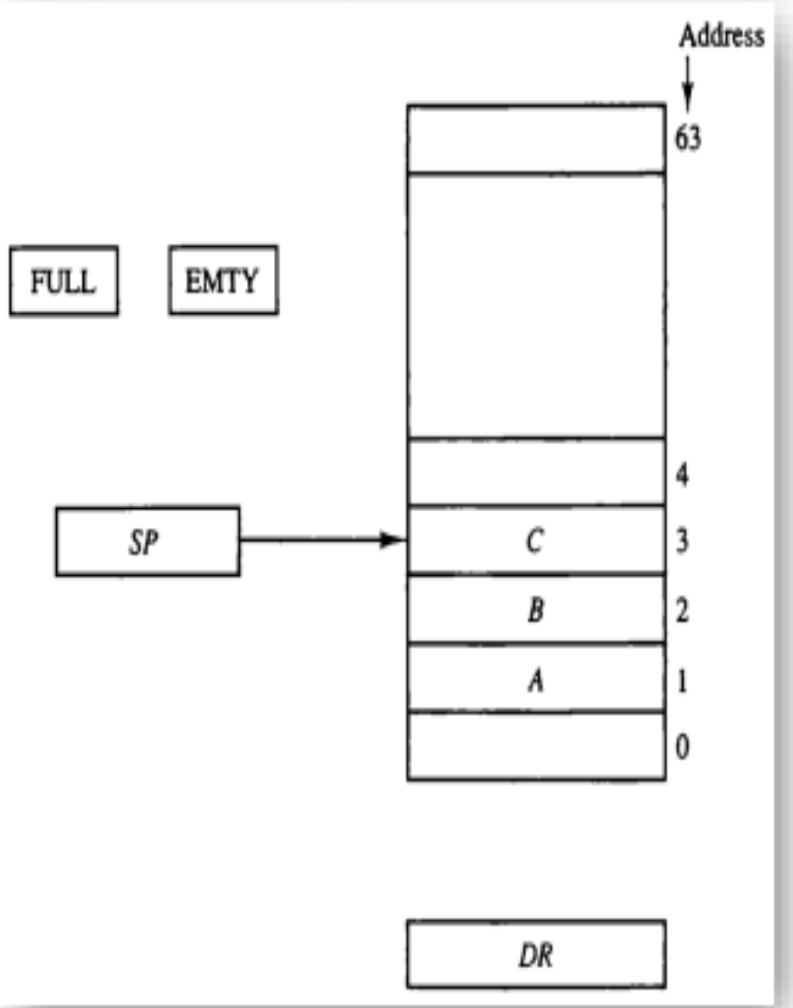
	Symbolic Designation				
Microoperation	SELA	SELB	SELD	OPR	Control Word
$R1 \leftarrow R2 - R3$	R2	R3	R1	SUB	010 011 001 00101
$R4 \leftarrow R4 \vee R5$	R4	R5	R4	OR	100 101 100 01010
$R6 \leftarrow R6 + 1$	R6	—	R6	INCA	110 000 110 00001
$R7 \leftarrow R1$	R1	—	R7	TSFA	001 000 111 00000
Output $\leftarrow R2$	R2	—	None	TSFA	010 000 000 00000
Output $\leftarrow$ Input	Input	—	None	TSFA	000 000 000 00000
$R4 \leftarrow$ shl $R4$	R4	—	R4	SHLA	100 000 100 11000

Stack Organization

- A **stack is a storage device that stores information** in such a manner that the item stored **last is the first item retrieved.**
- The **register that holds the address for the stack is called a stack pointer (SP)** because its value **always points at the top item in the stack.**
- The two operations of a stack are the insertion and deletion of items.
- The operation **of insertion is called push** (or push-down) because it can be thought of as the result of pushing a new item on top.
- The operation of **deletion is called pop** (or pop-up) because it can be thought of as the result of removing one item so that the stack pops up.
- **Stack Organization: 1) Register Stack 2) Memory Stack**

Register Stack Organization

- The organization of a 64-word register stack.
- The stack pointer register SP contains a binary number whose value is equal to the address of the word that is currently on top of the stack. Three items are placed in the stack: A, B, and C, in that order. Item C is on top of the stack so that the content of SP is now 3.
- To remove the top item, the stack is popped by reading the memory word at address 3 and decrementing the content of SP. Item B is now on top of the stack since SP holds address 2.
- To insert a new item, the stack is pushed by incrementing SP and writing a word in the next-higher location in the stack.



- In a 64-word stack, the stack pointer contains 6 bits because $2^6 = 64$. Since SP has only six bits, it cannot exceed a number greater than 63 (111111 in binary). When 63 is incremented by 1, the result is 0 since 111111 + 1 = 1000000 in binary, but SP can accommodate only the six least significant bits.

- Initially, SP is cleared to 0, EMTY is set to 1, and FULL is cleared to 0, so that SP points to the word at address 0 and the stack is marked empty and not full.

- **PUSH:** If the stack is not full (if FULL = 0), a new item is inserted with a push operation. The push operation is implemented with the following sequence of microoperations:

SP ← SP + 1	Increment stack pointer
M [SP] ← DR	Write item on top of the stack
If (SP = 0) then (FULL ← 1)	Check if stack is full
EMTY ← 0	Mark the stack not empty

- The stack pointer is incremented so that it points to the address of the next-higher word. A memory write operation inserts the word from DR into the top of the stack.

- **POP:** A new item is deleted from the stack if the stack is not empty (if EMTY = 0). The pop operation consists of the following sequence of microoperations:

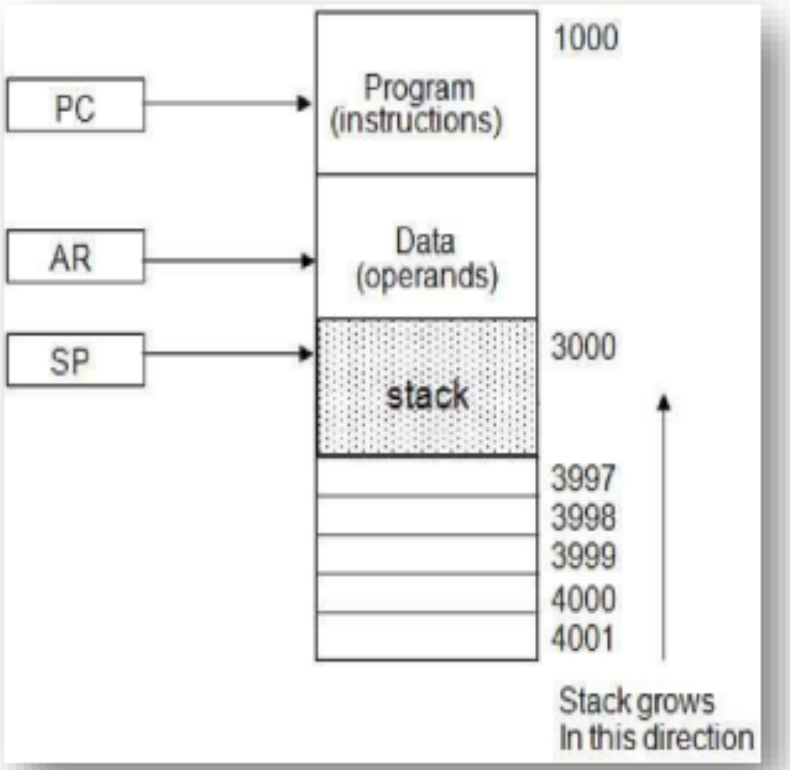
DR ← M [SP]	Read item from the top of stack
SP ← SP – 1	Decrement stack pointer
If (SP = 0) then (EMTY ←1)	Check if stack is empty
FULL ← 0	Mark the stack not full

- The top item is read from the stack into DR . The stack pointer is then decremented. If its value reaches zero, the stack is empty, so EMTY is set to 1.

Cont...

Memory Stack Organization

- The implementation of a stack in the CPU is done by assigning a portion of memory to a stack operation and using a processor register as a stack pointer.
- Computer memory is partitioned into three segments: program, data, and stack.
- The program counter PC points at the address of the next instruction in the program
- The address registers AR points at an array of data which is used during the execute phase to read an operand.
- The stack pointer SP points at the top of the stack which is used to push or pop items into or from the stack.
- The initial value of SP is 4001 and the stack grows with decreasing addresses. Thus the first item stored in the stack is at address 4000, the second item is stored at address 3999, and the last address that can be used for the stack is 3000.
- Assume that the items in the stack communicate with a data register DR.



Cont...

Memory Stack Organization

- **PUSH: Insert new item**

SP ← SP – 1	The stack pointer is decremented so that it points at the address of the next word.
M[SP] ← DR	A memory write operation inserts the word from DR into the top of the stack.

- **POP: Delete new item**

DR ← M[SP]	The top item is read from the stack into DR.
SP ← SP + 1	The stack pointer is then incremented to point at the next item in the stack.

Reverse Polish Notation (RPN)

- The postfix RPN notation, referred to as **Reverse Polish Notation (RPN), places the operator after the operands.**
- The following examples demonstrate the three representations:

A + B	Infix notation
+ A B	Prefix or Polish notation
A B +	Postfix or reverse Polish notation

- The reverse Polish notation is in a form suitable for stack manipulation. The expression

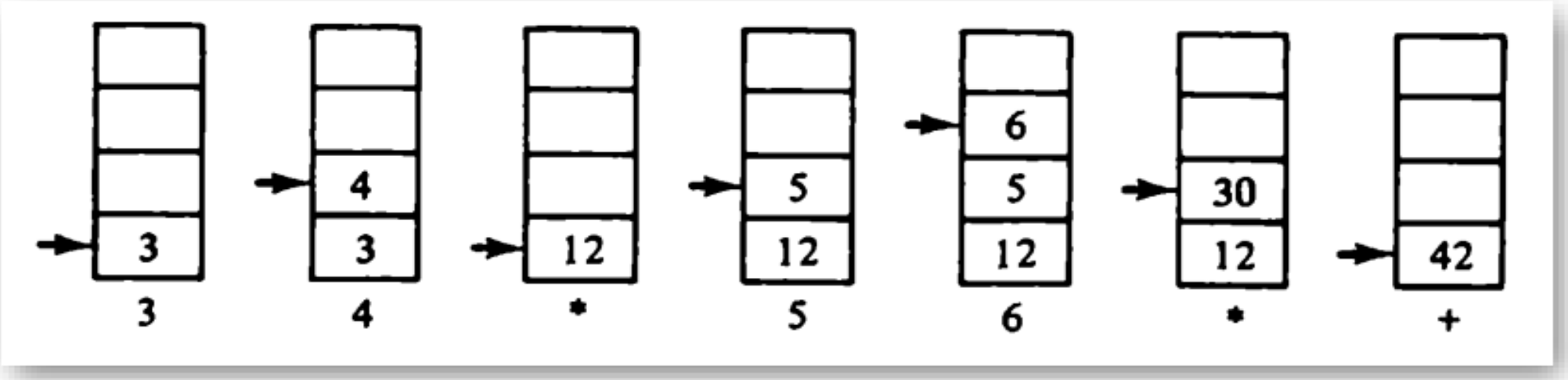
  A * B + C * D is written in reverse Polish notation as A B * C D * +

Reverse Polish Notation (RPN)

- **Evaluation of Arithmetic Expressions**

- (3 * 4) + (5 * 6) ⇨ 3 4 * 5 6 * +

- The stack operation for the following arithmetic expression,



- First the number 3 is pushed into the stack, then the number 4. The next symbol is the multiplication operator. This causes a multiplication of the two topmost items in the stack. The stack is then popped and the product is placed on top of the stack, replacing the two original operands. Next we encounter the two operands 5 and 6, so they are pushed into the stack. The stack operation that results from the next replaces these two numbers by their product. The last operation causes an arithmetic addition of the two topmost numbers in the stack to produce the final result of 42.

Instruction Formats

- Instructions are categorized into different formats with respect to the operand fields in the instructions.

1. Three Address Instructions
2. Two Address Instruction
3. One Address Instruction
4. Zero Address Instruction

Three address instructions

Opcode	Destination Address	Source Address 1	Source Address 2

Three-address instruction format

- **Three-address instruction format** has **three address fields.**
- **One address** field is used for **destination** and **two address** fields for **source.**
- Each **address field** may specify a **processor register or a memory operand.**
- It is also called **General register organization.**
- **Example Instruction:** **ADD R1, R2, R3** // **R1 ← R2 + R3**

  ADD R1, A, B // **R1 ← M[A] + M[B]**

Assembly Language Notation (ALN) — Equivalent — Register Transfer Notation (RTN)

Example: EVALUATE **C=A+B** (using Three address instruction

ADD C, A, B // **M[C] ← M[A]+M[B]**

Two address instructions

Two-address instruction format

- Two address instructions are most common in commercial computers
- It has **two address** fields
- Each **address field** may specify a processor register or a memory operand.
- **Example Instruction:** **ADD R1, R2** **// R1 ← R1 + R2**
  MOV R1, R2 **// R1 ← R2**

Example: EVALUATE **C=A+B** (using Two address

```
MOV C, A        // M[C] ← M[A]
ADD C, B        // M[C] ← M[C]+M[B]
```

One address instructions

One-address instruction format

- It has only **one address field**. The operand specified in the instruction may be source or destination, depending on instruction
- **One address instructions** use a **implied ACCUMULATOR (AC)** register for all data manipulation.
  - **Implied means** that the CPU already know that one operand is in accumulator so there is no need to specify it
- All operations done between **accumulator register** and **memory operand**.
- It is also called **Single Accumulator Organization**
- **Example Instruction: ADD X // AC ← AC + M[X]**

Example: EVALUATE C=A+B (using One address instructions)

- **LOAD A** // **AC ← M[A]**
- **ADD B** // **AC ← AC + M[B]**
- **STORE C** // **M[C] ← AC**

LOAD and STORE instruction: used for transfers to and from memory to AC register

- **LOAD instruction**: Operand (like A) specifies the source operand and the destination location is AC register. Eg: **LOAD A // AC ← M[A]**
- **STORE instruction**: Operand (like C) specifies the destination location and the source is AC register. Eg: **STORE C //M[C] ← AC**

Zero address instruction

Zero-address instruction format

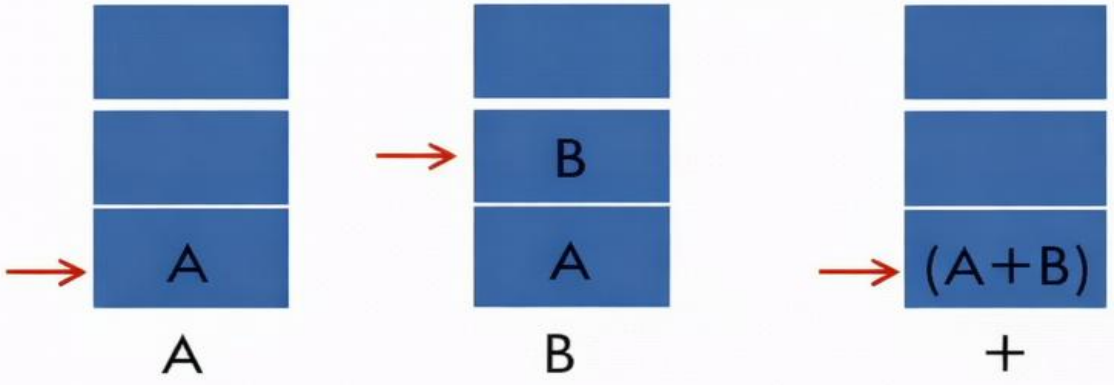
- In **zero address instructions** **Stack** is used. Also called **stack based organization**
- A **stack based computer** **does not use an address field** in the instructions (like **ADD**, **MUL**)
- To evaluate a arithmetic expression first it is converted to **Revere Polish Notation** i.e. **Post fix Notation.**
- The name "**zero-address**" is given to this type of computer of the absence of an address field in an instruction

Example: EVALUATE C=A+B (using zero address

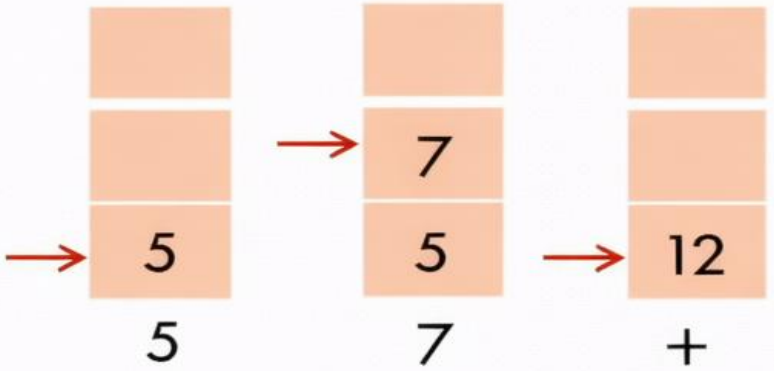
Postfix /Reverse Polish Notation: **AB+**

PUSH A	**// TOS ← A**
PUSH B	**// TOS ← B**
ADD	**// TOS ← (A+B)**
POP C	**// M[C] ← TOS**

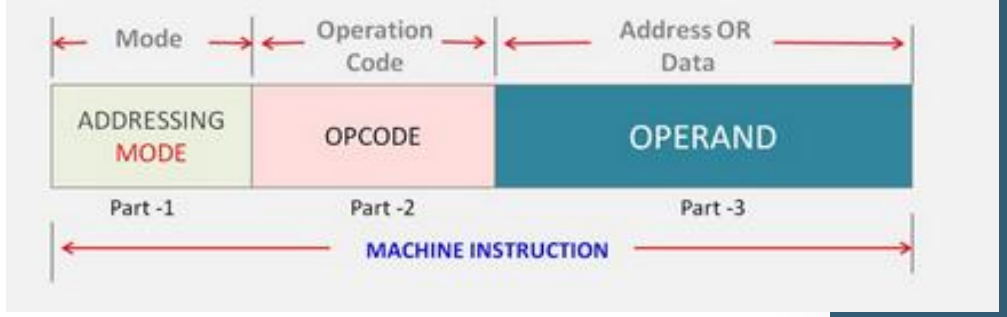
Stack Evaluation of A + B



Example: 5 + 7



Addressing Modes – (Types)



1. **Implied / Implicit** Mode
2. **Immediate** Mode

No address field is required

3. **Direct** Mode
4. **Indirect** Mode

Address field specifies Memory Location

5. **Register Direct** Mode
6. **Register Indirect** Mode
7. **Auto-Increment /Auto-Decrement** Mode

Address field specifies Processor Register

8. **Relative** Addressing Mode
9. **Indexed** Addressing Mode
10. **Base Register** Addressing Mode

Address field + content of CPU Register(There are called Displacement Addressing)

1. Implied (or Implicit) Mode

- **Operands** are **specified** **implicitly** in the definition of **instruction**

- **Examples:**
  - **CMA: Complement Accumulator**
    - **Operand** in **AC** is implied in the definition of the instruction
  - **CLA: Clear Accumulator**
    - **Operand** in **AC** is implied in the definition of the instruction
  - **PUSH**: Stack Push
    - **Operand** is implied to be on **top of the stack** (TOS)

- **All the register reference instructions that use Accumulator are implied mode,**
- **Zero address instruction in stack organization are also use implied mode.**

2. Immediate Mode

- **Operand** is specified in the **instruction itself.**

  An ***immediate mode instruction*** has an **operand field** rather than the **address field.**

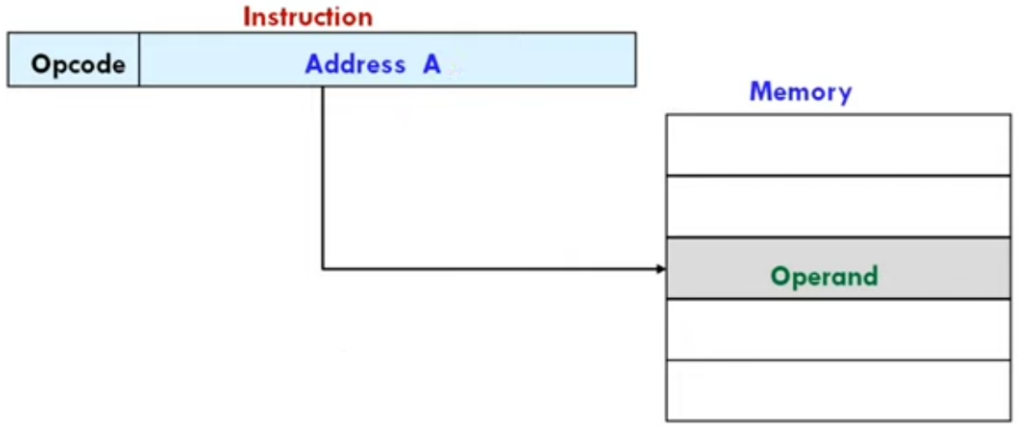
- **Operand field** contain the **actual operand**

- **Example:** LOAD #7 // AC ← 7

  ADD #5 // AC ← AC + 5

3. Direct Mode

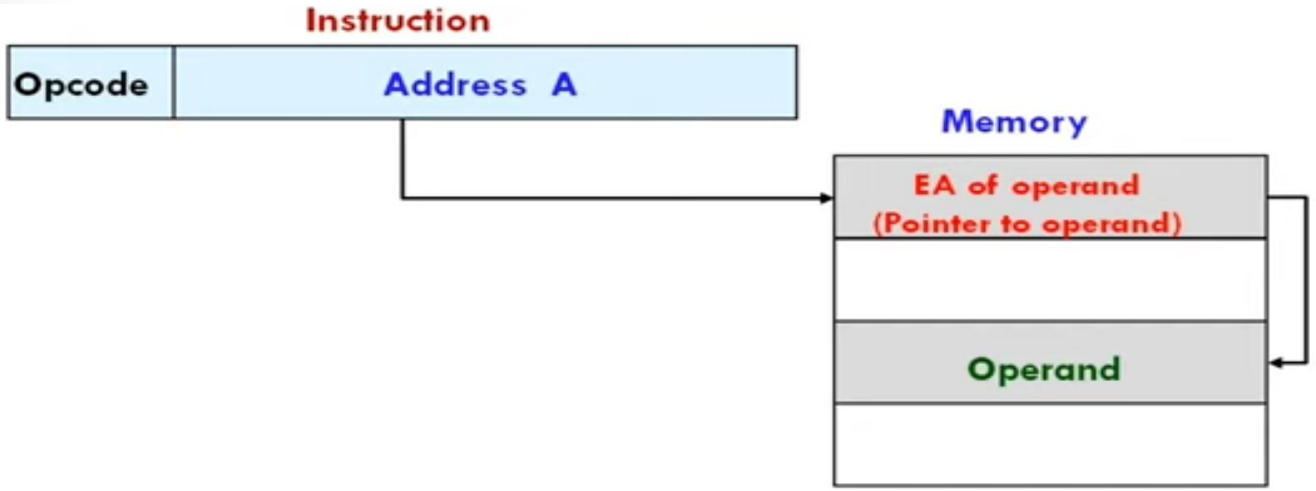
- The **operand** resides in **memory** and its **address** is given **directly** by the **address field** of instruction.
- **Address field** of instruction contains the **effective address EA** (or actual address) of **operand**
  - **EA = A** Effective address (EA) is equal to the address field (A)
- **Example:** **ADD A** // **AC ← AC + M[A]**
  LOAD B // **AC ← M[B]**
- It is also called as **absolute addressing mode**



4. Indirect Mode

- **Address field** of instruction specifies the **address** where ***the effective address of operand*** is **stored in memory**
  - **EA = (A)** or **EA = @A**

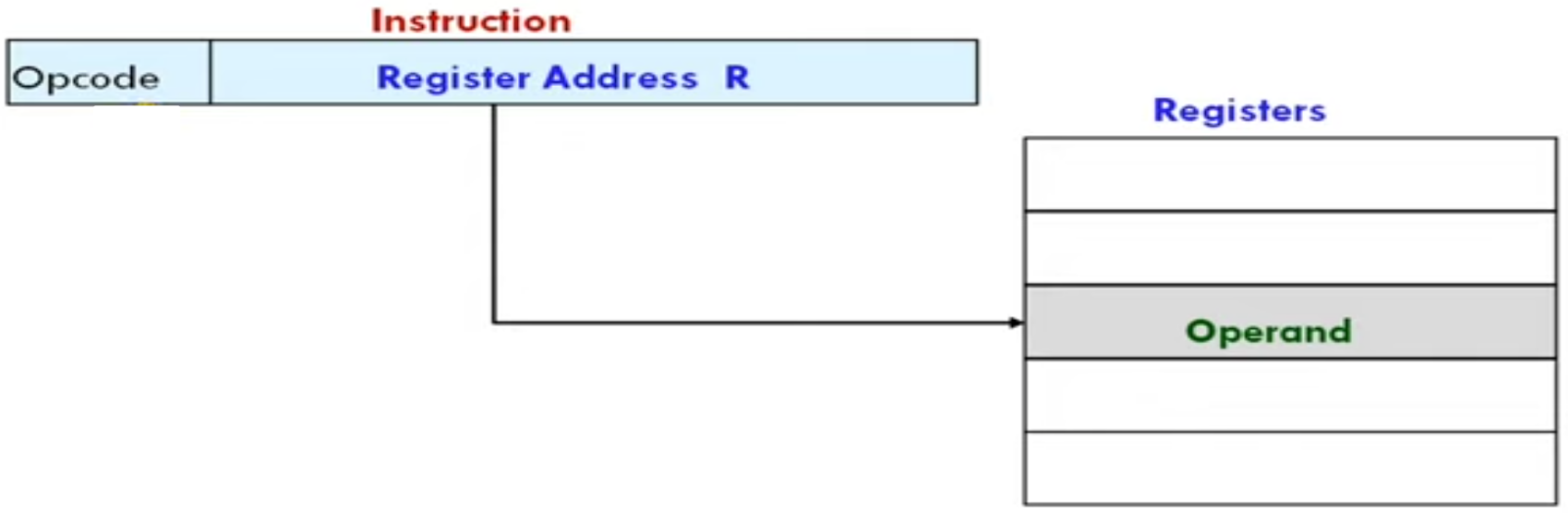
- **Example:** **ADD (A)** or **ADD @A** // AC ← AC + M[M[A]]



5. Register Mode

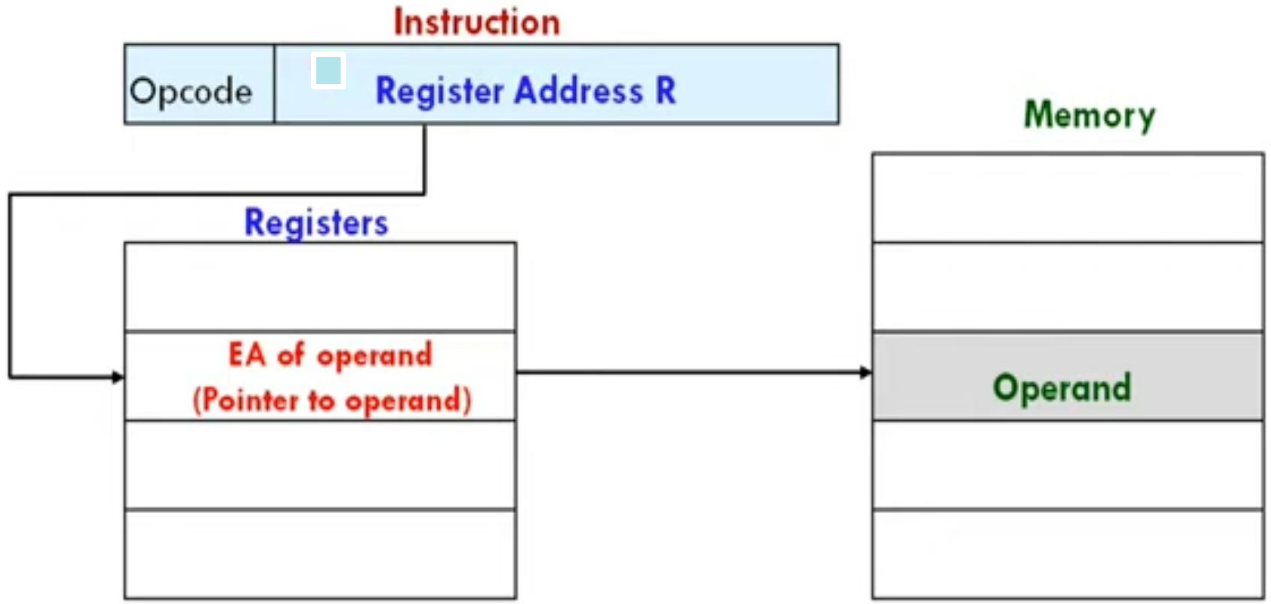
- **Operand** is stored in the **register.**
- **Address field** of the **instruction** refers to a **CPU register** that contains the **operand.**
  - EA = Ri

- **Example:** **MOV R1, R2** // R1 ← R2



6. Register Indirect Mode

- The address field of instruction specifies the register that contains the effective address of operand. The **operand** is in memory.
- The register contains the effective address of operand *rather than the operand itself.*
  - EA = (Ri)
- **Example:** **ADD (R1), R2** // M[R1] ← M[R1] + R2



7. Auto-increment/ Auto-decrement Mode

Similar to register indirect mode

except that

- In **Auto-increment**: The **register** is **incremented after** its value is used to access memory
  - EA = (**Ri**)+
- In **Autodecrement**:The **register** is **decrement before** its value is used to access memory
  - EA = –(**Ri**)

Example: (Autoincrement)

1. ADD R1, (R2)+ // R1 ← R1 + M[R2] , R2 ← R2 + 1

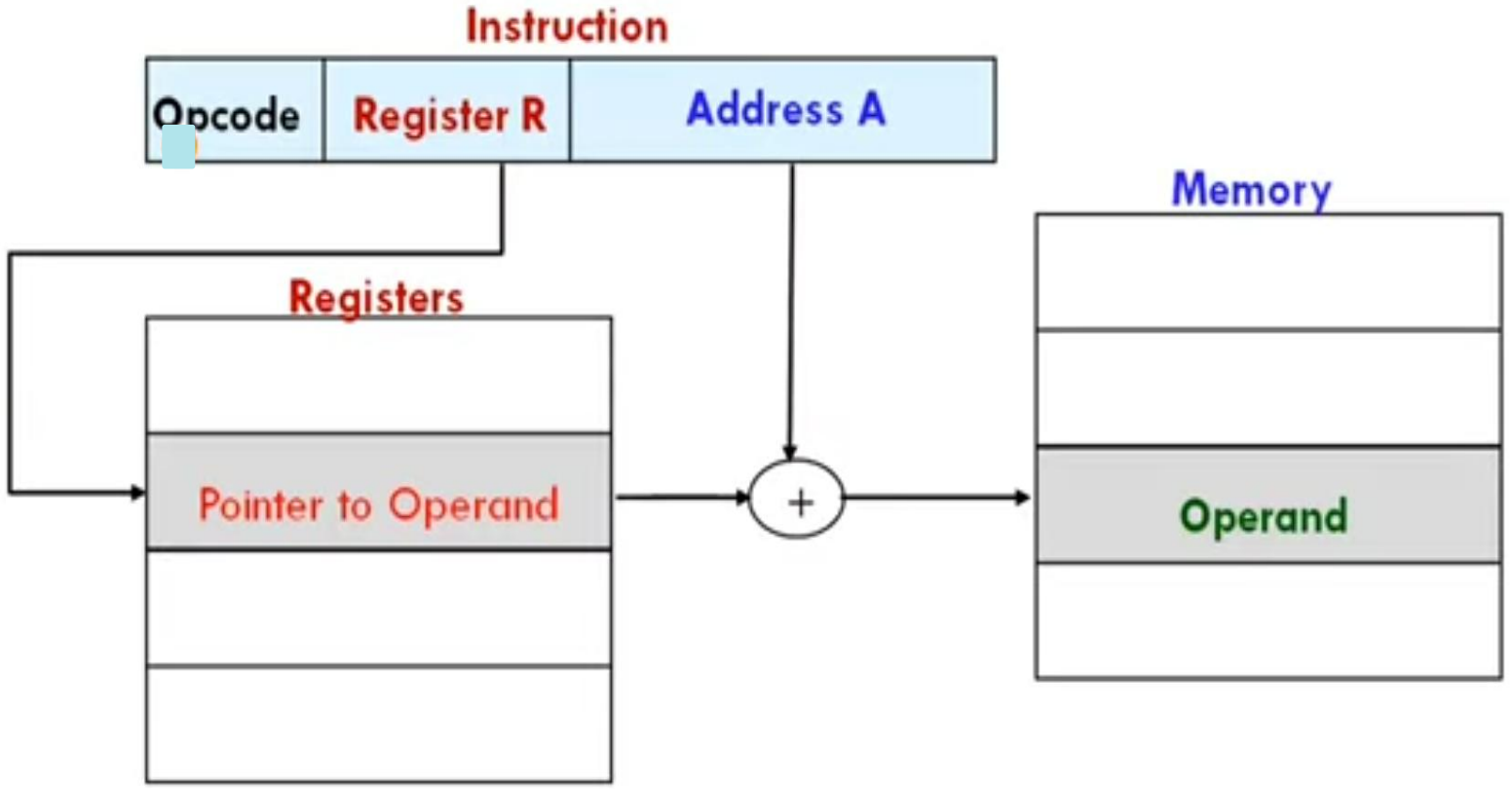
Example: (Autodecrement)

1. ADD R1, –(R2) // R2 ← R2 – 1 , R1 ← R1 + M[R2]

Displacement Addressing

Effective Address = Address part of instruction + content of CPU Register (PC/XR/R)

- EA = A + (R)



3 versions of displacement addressing:

1. Relative addressing (EA=PC+A)
2. Indexed addressing (EA=A+XR)
3. Base Register addressing (EA=Ri+A)

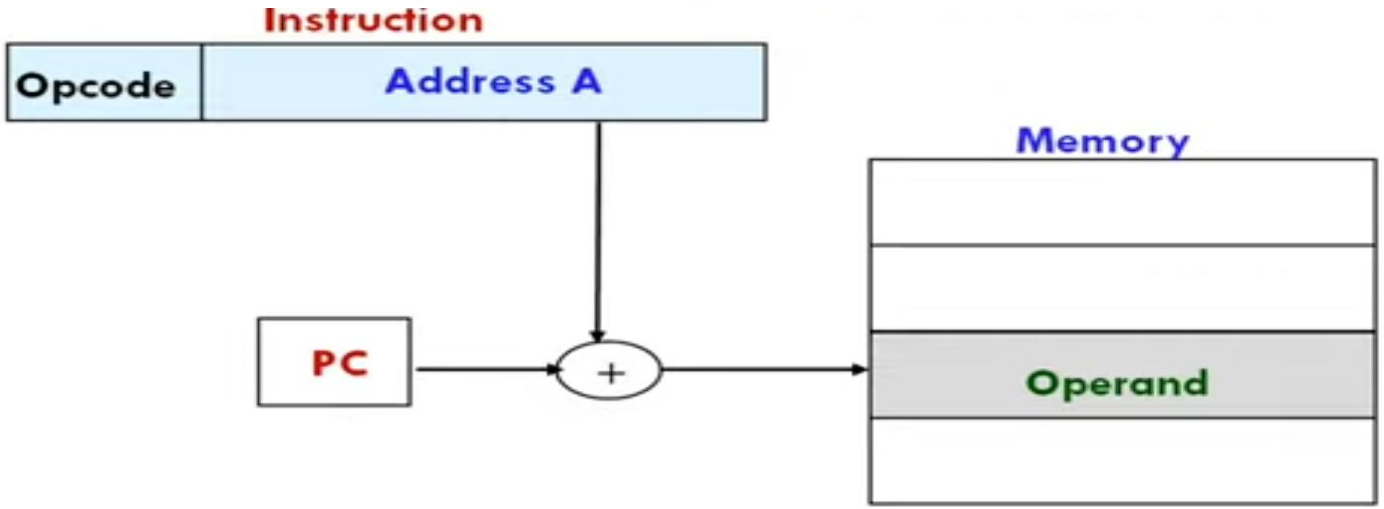
8. Relative Addressing Mode

- Register = **Program Counter (PC)**
- The content of **PC** is **added** to the **address part** of the instruction to **obtain** the **effective address of operand**

Effective Address = Content of PC + Address part of instruction

- **EA = (PC) + A**
- **Example:**

LOAD $A or **LOAD A(PC)** // **AC ← M[PC + A]**



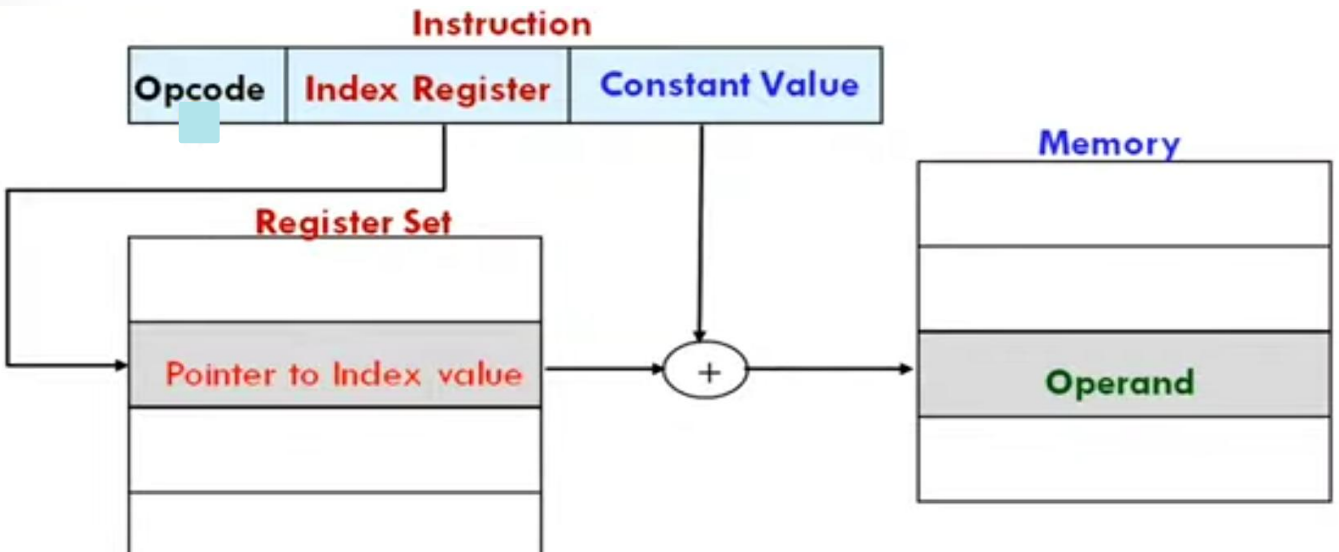
9. Indexed Addressing Mode

- Register = **Index Register**
- The content of **Index Register** is **added** to the **address part** of the instruction to **obtain** the **effective address of operand**

Effective Address = Content of XR + Address part of instruction

- **EA = A + (XR)** or **EA = A + (Ri)**
- **Example:** ADD R1, 20(R1) // R1 ← R1 + M[20 + R1]

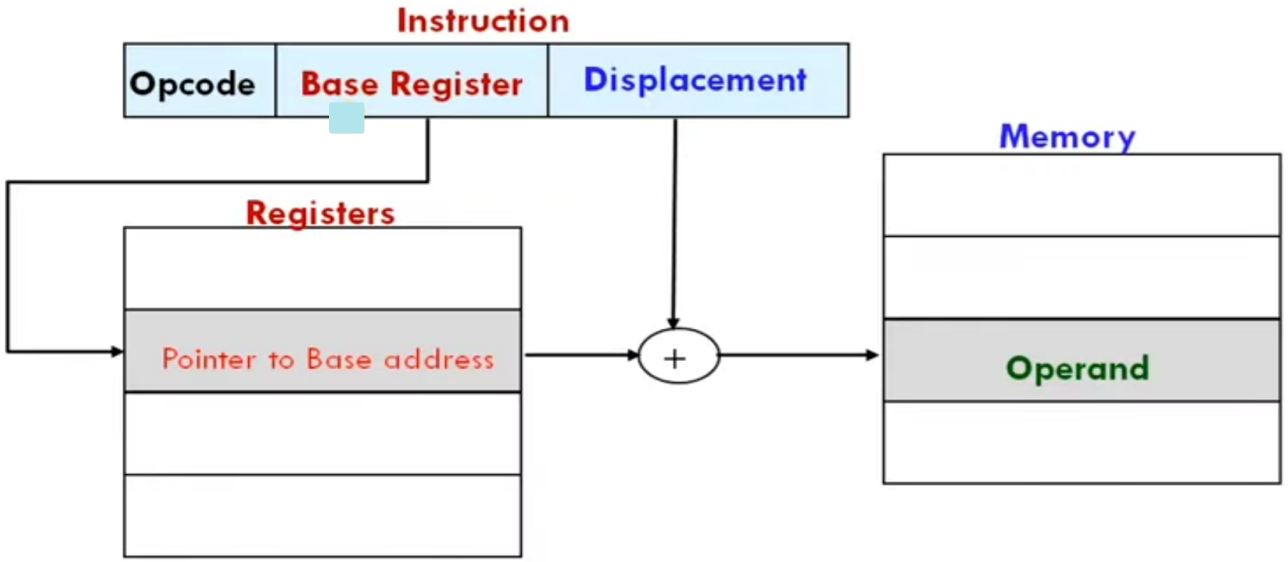
  ADD R1, 100(XR) // R1 ← R1 + M[100 + XR]



10. Base Register Addressing

- Similar to indexed addressing except that the register is now called a base register
  - **Register = Base Register**
- The content of **Base Register** is **added** to the **address part** of the instruction to **obtain** the **effective address of operand**

Effective Address = Content of BR + Address part of instruction



QUESTIONS ASKED IN GTU EXAM

Q.1 Enlist various kinds of addressing modes. Explain any five of same and support your answer by taking small example.
Q.2 Write two address, one address and zero address instructions program for the following arithmetic expression: X = (A + B) * (C – D / E) + F * G
Q.3 Explain status bit conditions with neat diagram.
Q.4 State the differences between RISC and CISC.
Q.5 Summarize following addressing modes with example. 1) Implied mode 2) Register mode
Q.6 Criticize Three-Address Instructions and Zero address instruction with common example.
Q.7 Differentiate RISC and CISC architecture.
Q.8 What is difference between two address and three address instructions?
Q.9 What is difference between direct and indirect addressing mode?
Q.10 What addressing mode means? Explain any three addressing modes in detail with example.
Q.11 Explain register stack and memory stack.
Q.12 Write a program to evaluate the arithmetic statement: A*B+C*D+E i. Using a stack organized computer.
Q.13 List down six major characteristics of RISC processors.
Q.14 Enlist different status bit conditions.
Q.15 What is addressing mode? Explain direct and indirect addressing mode with example.
Q.16 Compare and contrast RISC and CISC.